Lab 1: Introduction to ISA using SimpleScalar

Due by the end of Thursday, Feb. 19

In this exercise, you will run some Benchmarks to find out the distribution of instructions they execute. You will use *sim-profile* simulator available in the SimpleScalar toolset for this purpose. In the first part of this lab, you will use SimpleScalar configured for **ALPHA ISA** while in the second part, you will use both **ALPHA and PISA configuration**. If you want to run other benchmarks, they are available at www.simplescalar.com in the Benchmarks section.
[Note: Results of simulation may vary over multiple runs and among students. This fact is stated at www.simplescalar.com. But the results will be consistent which means they will make sense.]

Part 1

Quick fact about sim-profile simulator

It is a dynamic instruction profiler which means it collects information about instructions as they are executed by the simulator. Note that it is a functional simulator and not a detailed timing simulator as *sim-outorder*.

All the simulators including *sim-profile* are available in the **xyz/simplesim-3.0** directory (**xyz** is your CSE directory). Go to **xyz/simplesim-3.0** directory and type the following to seek help about *sim-profile*.

./sim-profile –h

Help can also be invoked just by typing simulator name without any arguments.
Go through help. Now use this help information to execute the following four benchmarks available in this tar file. Download benchmarks.tar.gz from Canvas. Unzip and untar this file in a directory e.g**. /u/xyz/benchmark** directory.

The information on how to execute these benchmarks using *sim-safe* is available in the **README** file which is also in the **u/xyz/benchmark** directory. Replace *sim-safe* with *sim-profile* and add information using *sim-profile* help to fill out the following table

1) anagram: a program for finding anagrams for a phrase, based on a dictionary.
2) compress95: (SPEC) compresses and decompresses a file in memory.
3) go: Artificial intelligence; plays the game of go against itself.
4) cc1: (SPEC) limited version of gcc.

Benchmark	Total # of Instructions	Load %	Store %	Uncond Branch %	Cond Branch %	Integer Computation %	Floating pt Computation %
anagram.alpha							
go.alpha							
compress95.alpha							
cc1.alpha							

Now answer the following questions for each individual benchmark executed above

1) Is the benchmark memory intensive or computation intensive?
2) Is the benchmark mainly using integer or floating point computations?
3) What % of the instructions executed are conditional branches? Given this %, how many instructions on average does the processor execute between each pair of conditional branch instructions (do not include the conditional branch instructions)

Part 2

In this part, you will compare the **PISA and Alpha ISA** by executing the same benchmarks on the two configurations.

Since the configuration from part 1 is still **Alpha**, execute the following benchmarks available in the **u/xyz/simplesim-3.0/tests-alpha/bin/** directory using *sim-profile* and record the results in the following table.

1) test-math: performs various math computations mostly in integer and displays their result.
2) test-fmath: performs various math functions mostly in integer.
3) test-llong: performs computations in long format.
4) test-printf: displays various print statements.

ALPHA Benchmark	Total # of instructions	Load %	Store %	Uncond branch %	Cond branch %	Integer Compute %	Floating compute %
test-math							
test-fmath							
test-llong							
Test-printf							

Now change configuration to PISA by typing the following three commands in a sequence.

./make clean
./make config-pisa
./make

To change configuration back to Alpha, execute the same command sequence except that in the second command replace **config-pisa** by **config-alpha**.

Now execute the following benchmarks available in the **u/xyz/simplesim-3.0/tests-pisa/bin.little/** directory using *sim-profile* and record the results in the following table.

PISA Benchmark	Total # of instructions	Load %	Store %	Uncond branch %	Cond branch %	Integer Compute %	Floating compute %
test-math							
test-fmath							
test-llong							
test-printf							

Now compare the two ISAs using a plot (a Histogram is preferred). Use the programming language or tool you prefer (e.g., MATLAB, EXCEL, Python, etc.) to plot the histogram. What can you conclude about the two ISAs from the Histogram.

Submission

Write a lab report according to the format requirement in Canvas. Upload the electronic copy of your lab report in Canvas by the end of the due date.